

第2章 线性表

提纲

CONTENTS

2.1 线性表的定义

2.2 线性表的顺序存储结构

2.3 线性表的链式存储结构

2.4 顺序表和链表的比较

2.5 线性表的应用

2.6 STL中的线性表

2.1 线性表的定义

2.1.1 什么是线性表

线性表是具有相同特性的数据元素的一个有限序列。

- 所有数据元素类型相同。
- 线性表是有限个数据元素构成的。
- 线性表中数据元素与位置相关，即每个数据元素有唯一的序号。

线性表的逻辑结构表示

$(a_0, a_1, \dots, a_i, a_{i+1}, \dots, a_{n-1})$

用图形表示的逻辑结构:



说明

线性表中每个元素 a_i 的唯一位置通过序号或者索引 i 表示，为了算法设计方便，将逻辑序号和存储序号统一，均假设从0开始，这样含 n 个元素的线性表的元素序号 i 满足 $0 \leq i \leq n-1$ 。

2.1.2 线性表的抽象数据类型描述

ADT List

{

数据对象:

$D = \{a_i \mid 0 \leq i \leq n-1, n \geq 0\}$

数据关系:

$r = \{ \langle a_i, a_{i+1} \rangle \mid a_i, a_{i+1} \in D, i = 0, \dots, n-2 \}$

基本运算:

CreateList(a): 由整数数组 a 中的全部元素建立线性表的相应存储结构。

Add(e): 将元素 e 添加到线性表末尾。

getlength(): 求线性表的长度。

GetElem(int i): 求线性表中序号为 i 的元素。

SetElem(int i , T e): 设置线性表中序号 i 的元素值为 e 。

GetNo(T e): 求线性表中第一个值为 e 的元素的序号。

Insert(int i , T e): 在线性表中插入数据元素 e 作为第 i 个元素。

Delete(int i): 在线性表中删除第 i 个数据元素。

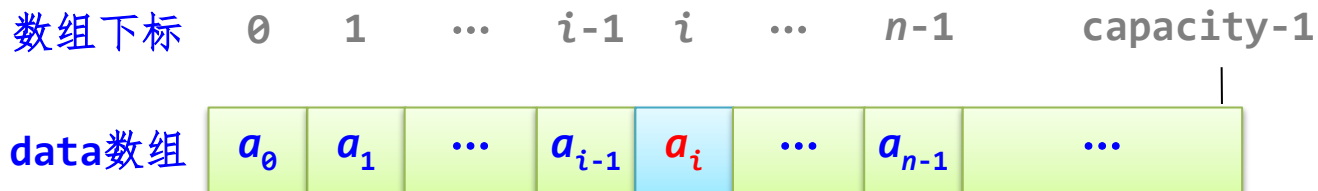
DispList(): 输出线性表的所有元素。

}

2.2 线性表的顺序存储结构

2.2.1 线性表的顺序存储结构—顺序表

长度为 n 的线性表存放在顺序表中



- **data**数组存放线性表元素
- **data**数组的容量（存放最多的元素个数）为**capacity**。
- 线性表中实际数据元素个数**length**

```
const int initcap=5;           //顺序表的初始容量（5）
template <typename T>
class SqList                   //顺序表类模板
{
public:
    T* data;                   //存放顺序表元素空间的指针
    int capacity;              //顺序表的容量
    int length;                //存放顺序表的长度
    //线性表的基本运算算法
};
```

2.2.2 线性表基本运算算法在顺序表中的实现

在动态分配顺序表的空间时，初始容量设置为**initcapacity**，当添加或者插入元素可能需要扩大容量，在删除元素时可能需要减少容量。

```
void recap(int newcap)                                //改变顺序表的容量为newcap
{
    if (newcap<=0) return;
    T* olddata=data;
    data=new T[newcap];                                //分配新空间
    capacity=newcap;                                   //更新容量
    for(int i=0;i<length;i++)                          //元素复制
        data[i]=olddata[i];
    delete [] olddata;                                  //释放原空间
}
```

1. 整体建立顺序表

由含若干个元素的数组 a 的全部元素整体创建顺序表，即依次将 a 中的元素添加到 $data$ 数组的末尾，当出现上溢出时按实际元素个数 $length$ 的两倍扩大容量。

```
void CreateList(T a[],int n)           //由数组a中元素整体建立顺序表
{
    for (int i=0;i<n;i++)
    { if (length==capacity)             //容量不够时
        recap(2*length);               //扩大容量
      data[length]=a[i];
      length++;                         //添加后元素个数增加1
    }
}
```


2. 顺序表基本运算算法

(1) 顺序表的初始化和销毁

构造函数

```
SqList()  
{ data=new T[initcap];  
  capacity=initcap;  
  length=0;  
}
```

//构造函数
//为data分配初始容量大小的空间
//初始化容量
//初始时置length为0

初始化复制构造函数（拷贝构造函数）

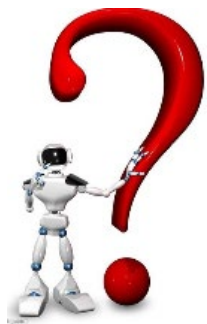
```
SqList(const SqList<T>& s)           //初始化复制构造函数
{
    capacity=s.capacity;              //复制容量
    length=s.length;                  //复制长度
    data=new T[capacity];              //为当前顺序表分配空间
    for (int i=0;i<length;i++)        //元素复制
        data[i]=s->data[i];
}
```

析构函数

```
~SqlList()           //析构函数  
{  
    delete [] data;    //释放data指向的空间  
}
```

(2) 将元素 e 添加的线性表末尾Add(e)

```
void Add(T e)                //在线性表的末尾添加一个元素e
{   if (length==capacity)    //顺序表空间满时倍增容量
    recap(2*length);
    data[length]=e;          //添加元素e
    length++;                //长度增1
}
```



时间复杂度是多少?

(3) 求线性表的长度getlength()

```
int Getlength()                //求顺序表的长度
{
    return length;
}
```

(4) 求线性表中序号为*i*的元素GetElem(*i*,&*e*)

```
bool GetElem(int i, T& e)    //求序号i的元素值
{  if (i<0 || i>=length)
    return false;           //参数错误时返回false
    e=data[i];              //取元素值
    return true;            //成功找到元素时返回true
}
```

(5) 设置线性表中序号为*i*的元素SetElem(*i*, *e*)

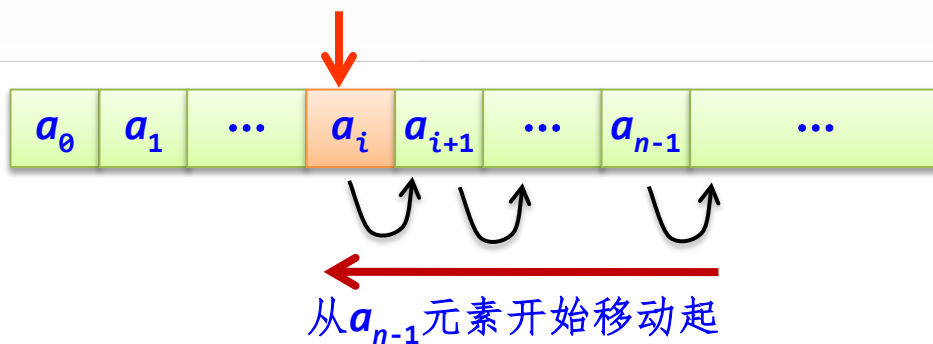
```
bool SetElem(int i,T e)           //设置序号i的元素值
{  if (i<=0 || i>=length)        //参数错误时返回false
    return false;
    data[i]=e;
    return true;
}
```

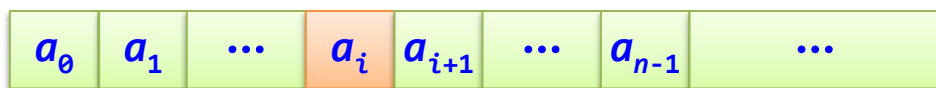
(6) 求线性表中第一个值为 e 的元素的逻辑序号GetNo(e)

```
int GetNo(T e)                                //查找第一个为 $e$ 的元素的序号
{
    int i=0;
    while(i<length && data[i]!=e)
        i++;
    if (i>=length)                             //查找元素 $e$ 
        return -1;                           //未找到时返回-1
    else
        return i;                             //找到后返回其序号
}
```


(7) 在线性表中插入 e 作为第 i 个元素 $\text{Insert}(i, e)$

```
bool Insert(int i, T e)           //在线性表中序号 $i$ 位置插入元素 $e$   
{ if (i<0 || i>length)          //参数 $i$ 错误返回false  
    return false;  
  
    if (length==capacity)        //满时倍增容量  
        recap(2*length);  
  
    for (int j=length; j>i; j--)  //data[i]及后元素后移一个位置  
        data[j]=data[j-1];  
  
    data[i]=e;                   //插入元素 $e$   
    length++;                   //长度增1  
    return true;  
}
```





主要时间花在元素移动上。有效插入位置 i 的取值是 $0 \sim n$ ，共有 $n+1$ 个位置可以插入元素：

- 当 $i=0$ 时，移动次数为 n ，达到最大值。
- 当 $i=n$ 时，移动次数为 0 ，达到最小值。
- 其他情况，需要移动 $\text{data}[i..n-1]$ 的元素，移动次数为 $(n-1)-i+1=n-i$ 。

$p_i = \frac{1}{n+1}$ 所需移动元素的平均次数为：

$$\sum_{i=0}^n p_i (n-i) = \frac{1}{n+1} \sum_{i=0}^n (n-i) = \frac{1}{n+1} \times \frac{n(n+1)}{2} = \frac{n}{2}$$

插入算法的平均时间复杂度为 $O(n)$ 。

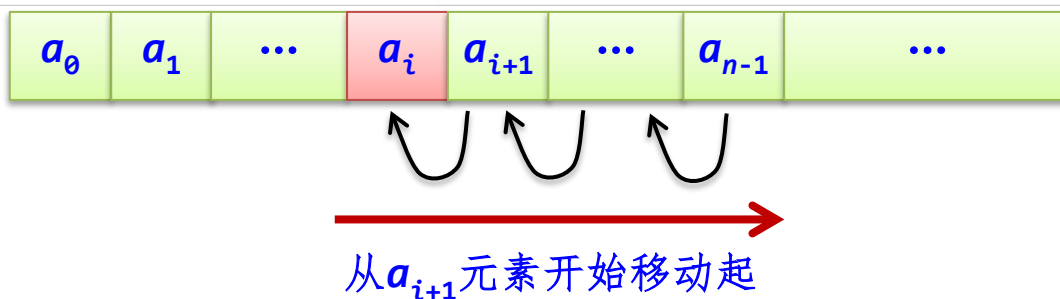


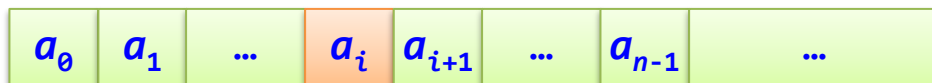
说明

扩容运算`recap()`在 n 次插入中仅仅调用一次，其平摊时间为 $O(1)$ ，上述算法时间分析中可以忽略它。

(8) 在线性表中删除第*i*个数据元素Delete(*i*)

```
bool Delete(int i)                                //在线性表中删除序号i的元素
{ if (i<0 || i>=length)                          //参数i错误返回false
    return false;
    for(int j=i;j<length-1;j++)
        data[j]=data[j+1];                        //将data[i]之后元素前移一个位置
    length--;                                       //长度减1
    if (capacity>initcap && length<=capacity/4)
        recap(capacity/2);                        //满足缩容条件则容量减半
    return true;
}
```





主要时间花在元素移动上。有效删除位置 i 的取值是 $0 \sim n-1$ ，共有 n 个位置可以删除元素：

- 当 $i=0$ 时，移动次数为 $n-1$ ，达到最大值。
- 当 $i=n-1$ 时，移动次数为 0 ，达到最小值。
- 其他情况，需要移动 $\text{data}[i+1..n-1]$ 的元素，移动次数为 $(n-1)-(i+1)+1=n-i-1$ 。

$p_i = \frac{1}{n}$ 所需移动元素的平均次数为：

$$\sum_{i=0}^{n-1} p_i (n - i - 1) = \frac{1}{n} \sum_{i=0}^{n-1} (n - i - 1) = \frac{1}{n} \times \frac{n(n-1)}{2} = \frac{n-1}{2}$$

删除算法的平均时间复杂度为 $O(n)$ 。

(9) 输出线性表所有元素DispList()

```
void DispList()  
{ for (int i=0;i<length;i++)  
    cout << data[i] << " ";  
  cout << endl;  
}
```

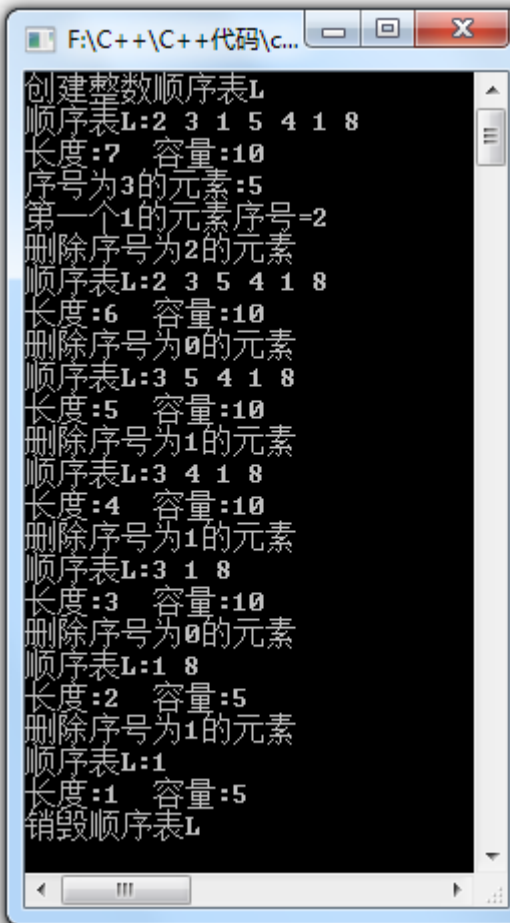
//输出顺序表L中所有元素
//遍历顺序表中各元素值

```
#include "SqList.cpp" //引用顺序表类
int main()
{   int i,e;
    SqList<int> L; //建立类型为int的顺序表对象L
    cout << "创建整数顺序表L" << endl;
    L.Insert(0,2); //插入元素2
    L.Insert(1,3); //插入元素3
    L.Insert(2,1); //插入元素1
    L.Insert(3,5); //插入元素5
    L.Insert(4,4); //插入元素4
    L.Insert(5,1); //插入元素1
    L.Add(8); //添加整数8
    cout << "顺序表L:"; L.DispList();
    cout << "长度:" << L.length << "   容量:"
        << L.capacity << endl;
    i=3; L.GetElem(i,e);
    cout << "序号为" << i << "的元素:" << e <<
    endl;
```



```
F:\C++\C++代码\c...
创建整数顺序表L
顺序表L:2 3 1 5 4 1 8
长度:7 容量:10
序号为3的元素:5
第一个1的元素序号=2
删除序号为2的元素
顺序表L:2 3 5 4 1 8
长度:6 容量:10
删除序号为0的元素
顺序表L:3 5 4 1 8
长度:5 容量:10
删除序号为1的元素
顺序表L:3 4 1 8
长度:4 容量:10
删除序号为1的元素
顺序表L:3 1 8
长度:3 容量:10
删除序号为0的元素
顺序表L:1 8
长度:2 容量:5
删除序号为1的元素
顺序表L:1
长度:1 容量:5
销毁顺序表L
```

```
e=1;
cout << "第一个" << e << "的元素序号=" <<
    L.GetNo(e) << "\n";
i=2; cout << "删除序号为" << i << "的元素\n";
L.Delete(i);
cout << "顺序表L:";L.DispList();
cout << "长度:" << L.length << " 容量:"
    << L.capacity << endl;
int b[]={0,1,1,0,1};
for (int i=0;i<5;i++)
{ cout << "删除序号为" << b[i] << "的元素\n";
  L.Delete(b[i]);
  cout << "顺序表L:";L.DispList();
  cout << "长度:" << L.length << " 容量:"
      << L.capacity << endl;
}
cout << "销毁顺序表L" << endl;
return 0;
}
```

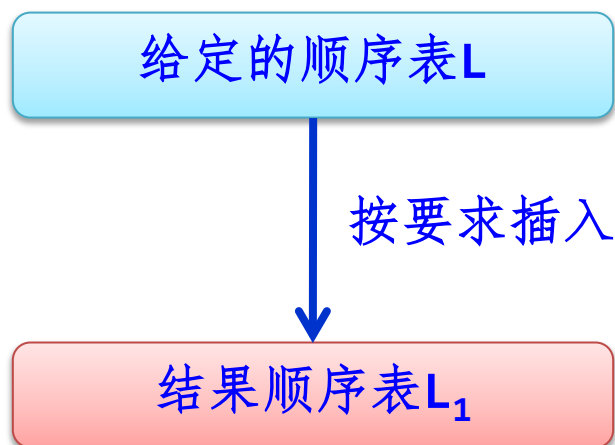


```
F:\C++\C++代码\c...
创建整数顺序表L
顺序表L:2 3 1 5 4 1 8
长度:7 容量:10
序号为3的元素:5
第一个1的元素序号=2
删除序号为2的元素
顺序表L:2 3 5 4 1 8
长度:6 容量:10
删除序号为0的元素
顺序表L:3 5 4 1 8
长度:5 容量:10
删除序号为1的元素
顺序表L:3 4 1 8
长度:4 容量:10
删除序号为1的元素
顺序表L:3 1 8
长度:3 容量:10
删除序号为0的元素
顺序表L:1 8
长度:2 容量:5
删除序号为1的元素
顺序表L:1
长度:1 容量:5
销毁顺序表L
```


2.2.3 顺序表的应用算法设计示例

1. 基于顺序表基本操作的算法设计（自学）

2. 基于整体建立顺序表的算法设计



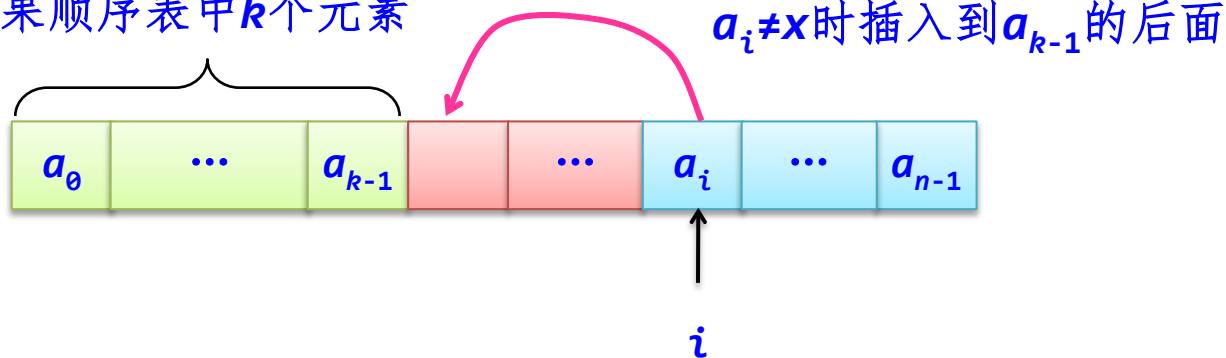
如果两者可以共享，直接在L中操作产生结果顺序表

【例2.3】对于含有 n 个整数元素的顺序表 L ，设计一个算法用于删除其中所有值为 x 的元素。

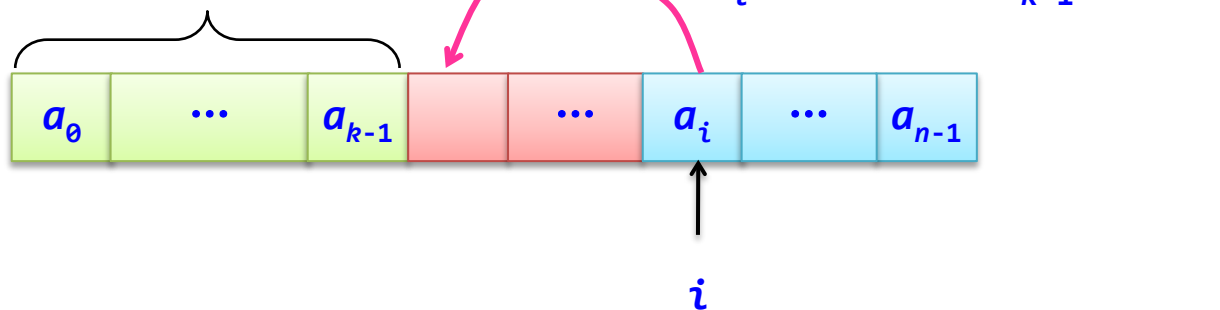
例如 $L=(1, 2, 1, 5, 1)$ ，若 $x=1$ ，删除后 $L=(2, 5)$ 。并给出算法的时间复杂度和空间复杂度。

解法1: 对于整数顺序表L，删除其中所有x元素后得到的结果顺序表可以与原L共享，所以求解问题转化为新建结果顺序表。

结果顺序表中 k 个元素



结果顺序表中 k 个元素



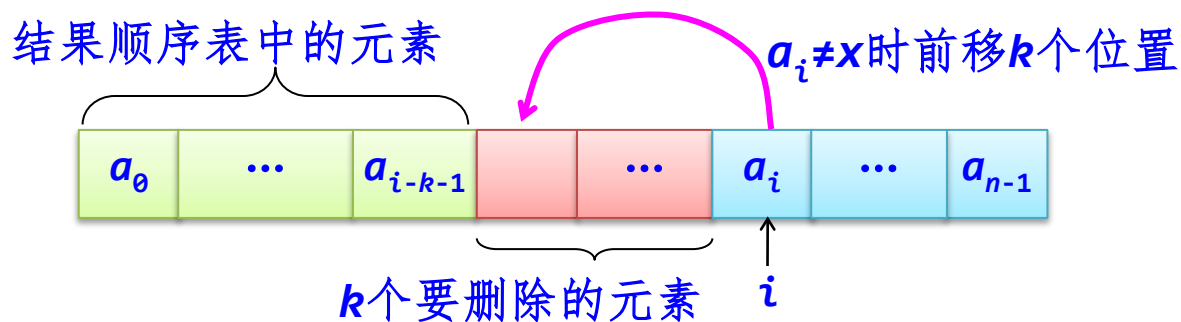
```
template <typename T>
void Deletex1(SqList<T>& L, int x) //求解算法1
{
    int k=0;
    for (int i=0;i<L.length;i++)
        if (L.data[i]!=x) //将不为x的元素插入到data中
        {
            L.data[k]=L.data[i];
            k++;
        }
    L.length=k; //重置L的长度为k
}
```

解法2：前移法，对于整数顺序表L，从头开始遍历L，用 k 累计当前为止值为 x 的元素个数（初始值为0），处理当前序号为 i 的元素 a_i ：

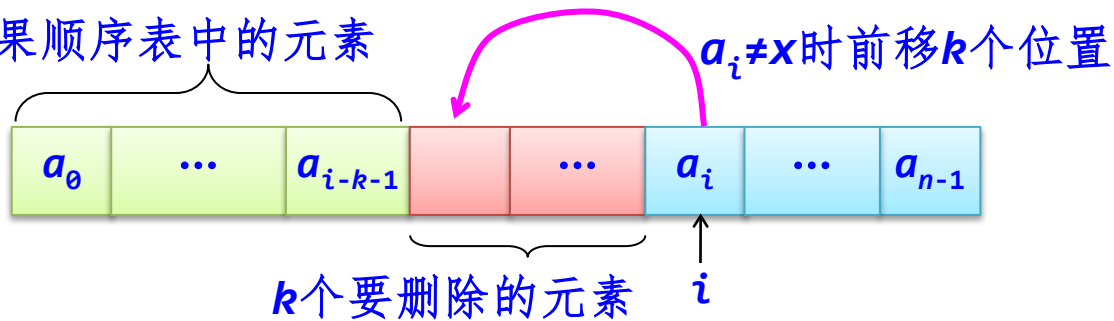
（1）若 a_i 是不为 x 的元素，此时前面有 k 个为 x 的元素，将 a_i 前移 k 个位置，继续处理下一个元素。

（2）若是为 x 的元素，置 $k++$ ，继续处理下一个元素。

最后将L的长度减少 k 。



结果顺序表中的元素



```
template <typename T>
void Deletex2(SqList<T>& L, int x) //求解算法2
{ int k=0; //累计等于x的元素个数
  for (int i=0;i<L.length;i++)
    if (L.data[i]!=x) //将不为x的元素前移k个位置
      L.data[i-k]=L.data[i];
    else //累计删除的元素个数k
      k++;
  L.length-=k; //将L的长度减少k
}
```

```

template <typename T>
void Deletex1(SqList<T>& L, int x) //求解算法1
{
    int k=0;
    for (int i=0;i<L.length;i++)
    {
        if (L.data[i]!=x) //将不为x的元素插入到data中
        {
            L.data[k]=L.data[i];
            k++;
        }
        L.length=k; //重置L的长度为k
    }
}

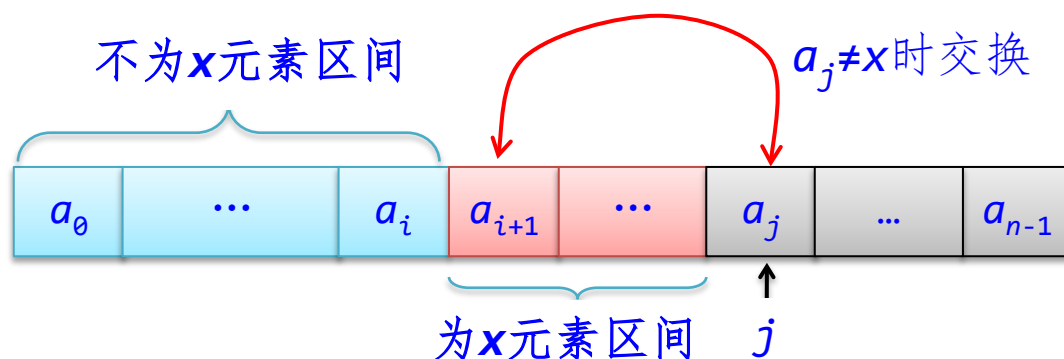
```

```

template <typename T>
void Deletex2(SqList<T>& L, int x) //求解算法2
{
    int k=0; //累计等于x的元素个数
    for (int i=0;i<L.length;i++)
    {
        if (L.data[i]!=x) //将不为x的元素前移k个位置
        {
            L.data[i-k]=L.data[i];
        }
        else //累计删除的元素个数k
        {
            k++;
        }
        L.length-=k; //将L的长度减少k
    }
}

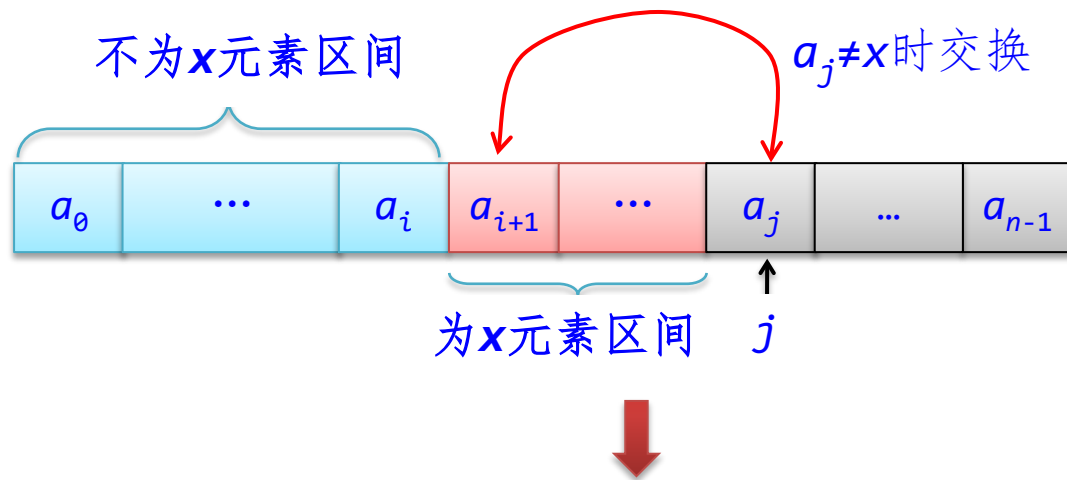
```

解法3：由解法2延伸出区间划分法



初始时，“不为 x 的区间”为空 $\Rightarrow i=-1$ ， j 从 0 开始遍历，
“为 x 的区间”是 $a[i+1..j-1]$

- 若 $a[j]=x$ ，跳过， $j++$ 。
- 若 $a[j] \neq x$ ，操作是，先执行 $i++$ ，将 $a[j]$ 与 $a[i]$ 进行交换，再执行 $j++$ 继续遍历其余元素。



```

template <typename T>
void Deletex3(SqList<T>& L, int x)           //求解算法3
{
    int i=-1, j=0;
    while (j<L.length)
    {
        if (L.data[j]!=x)
        {
            i++;
            if (i!=j)
                swap(L.data[i], L.data[j]); //序号i和j的两个元素交换
        }
        j++; //继续遍历
    }
    L.length=i+1; //将L的长度置为i+1
}

```

各种顺序表的高效算法设计

- 设计一个算法，从一给定的顺序表L中删除元素值在 x 到 y ($x \leq y$) 之间的所有元素，要求算法的时间复杂度为 $O(n)$ ，空间复杂度为 $O(1)$ 。
- 设计一个算法从有序顺序表中删除重复的元素，并使剩余元素间的相对次序保持不变。
- ...

3. 有序顺序表的算法设计

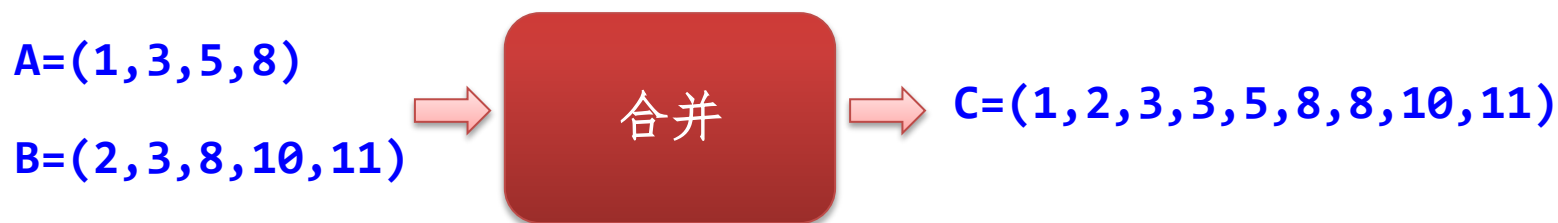
- **有序表**是指按元素值或者某属性值递增或者递减排列的线性表，有序表是线性表的一个子集。
- **有序顺序表**是有序表的顺序存储结构。
- 对于有序表可以利用其元素的有序性提高相关算法的效率，**二路归并**就是有序表的一种经典算法。

$A = (1, 3, 8, 23, 30)$

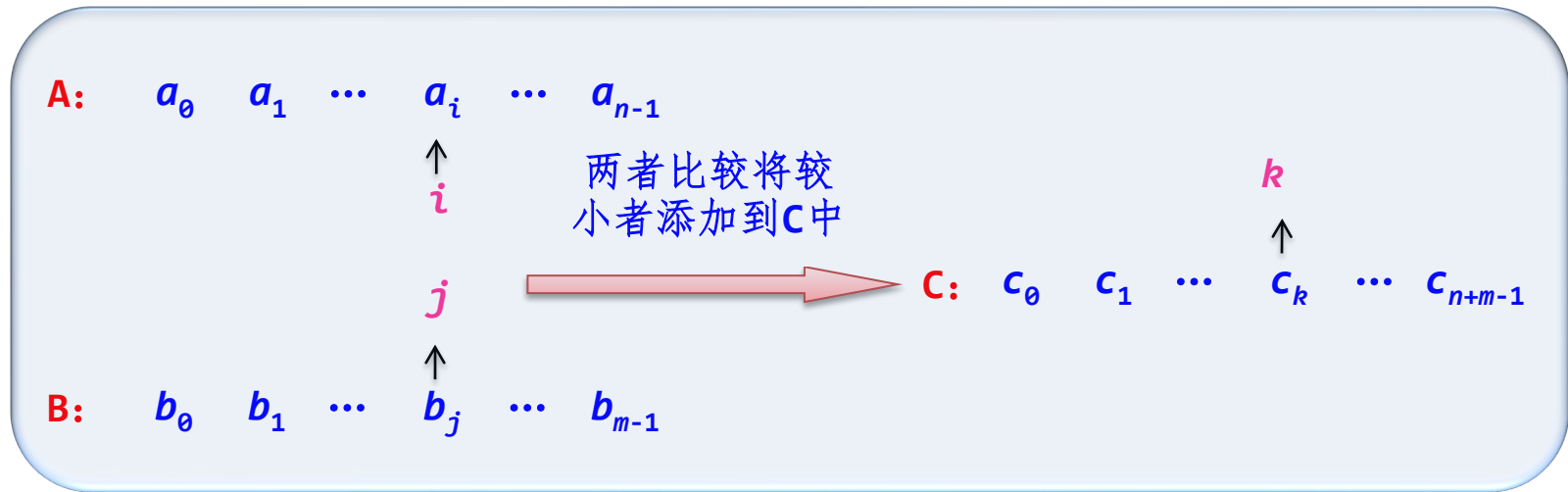


一个递增有序整数顺序表

【例2.4】 有两个按元素值递增有序的整数顺序表A和B，设计一个算法将顺序表A和B的全部元素合并到一个递增有序顺序表C中。并给出算法的时间复杂度和空间复杂度。



二路归并:



i 遍历A, j 遍历B, 均从0开始

while i, j 都没有超界

a_i 与 b_j 比较: 较小元素添加到C中, 后移相应指针

将没有遍历完的元素添加到C中

```

template <typename T>
void Merge2(SqList<T> A, SqList<T> B, SqList<T>& C)
{  int i=0, j=0;                //i用于遍历A, j用于遍历B
  while (i<A.length && j<B.length) //两个表均没有遍历完毕
  {  if (A.data[i]<B.data[j])
      {  C.Add(A.data[i]);        //归并A[i]:将较小的A[i]添加到C中
        i++;
      }
    else                          //归并B[j]:将较小的B[j]添加到C中
    {  C.Add(B.data[j]);
      j++;
    }
  }
}

```

```
while (i<A.length)
{   C.Add(A.data[i]);
    i++;
}

while (j<B.length)
{   C.Add(B.data[j]);
    j++;
}
}
```

//若A没有遍历完毕
//归并A中剩余元素

//若B没有遍历完毕
//归并B中剩余元素

- 算法中尽管有多个**while**循环语句，但恰好对顺序表**A**、**B**中每个元素均访问一次，所以时间复杂度为 **$O(n+m)$** 。
- 算法中需要在临时顺序表**C**中添加 **$n+m$** 个元素，所以算法的空间复杂度也是 **$O(n+m)$** 。

2.3 线性表的链式存储结构

2.3.1 线性表的链式存储结构—链表

线性表：

$(a_0, a_1, \dots, a_i, a_{i+1}, \dots, a_{n-1})$

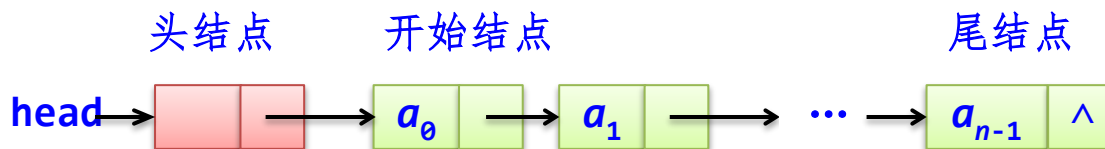


映射

结点

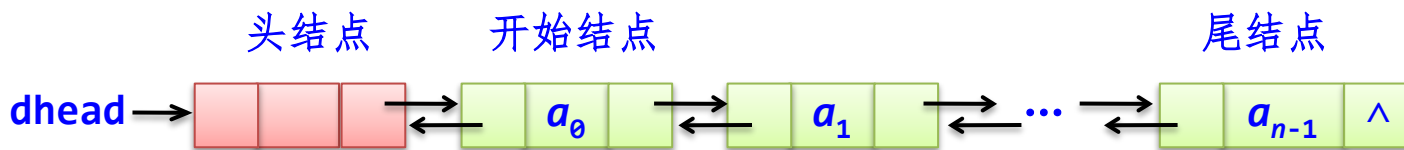
结点包含有元素值和前后继结点的地址信息

- 如果每个结点只设置一个指向其后继结点的指针成员，这样的链表称为线性单向链接表，简称**单链表**。



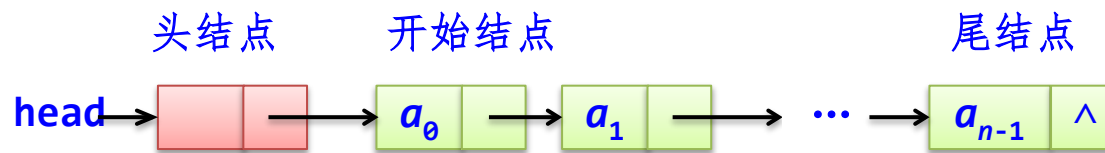
(a) 单链表

- 如果每个结点中设置两个指针成员，分别用以指向其前驱结点和后继结点，这样的链表称之为线性双向链接表，简称**双链表**。



(b) 双链表

2.3.2 单链表



每个结点为**LinkNode**类型，包括存储元素的数据属性**data**和存储后继结点的指针**next**。



```
template <typename T>
struct LinkNode                                //单链表结点类型
{
    T data;                                    //存放数据元素
    LinkNode<T>* next;                        //下一个结点的指针

    LinkNode():next(NULL) {}                  //构造函数
    LinkNode(T d):data(d),next(NULL) {}       //重载构造函数
};
```

单链表类模板LinkedList

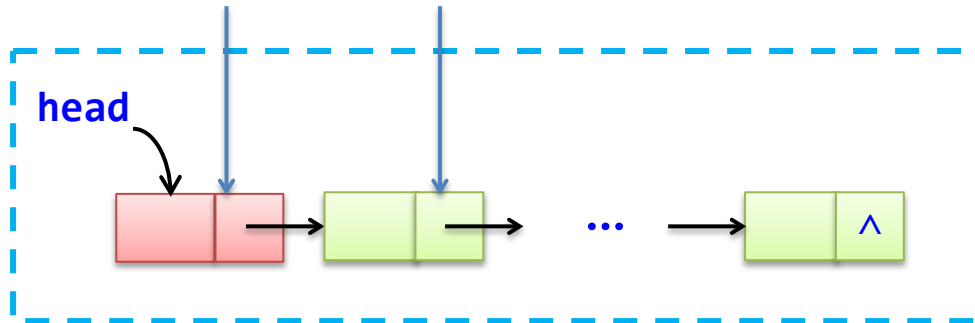
```
template <typename T>
class LinkedList                                //单链表类模板
{
public:
    ListNode<T>* head;                          //单链表头结点
    //基本运算算法
};
```



结点引用方式:

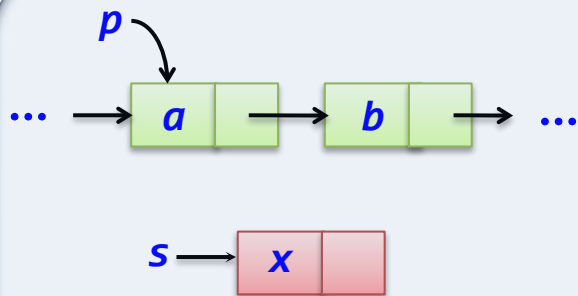
L.head L.head->next

单链表对象L:

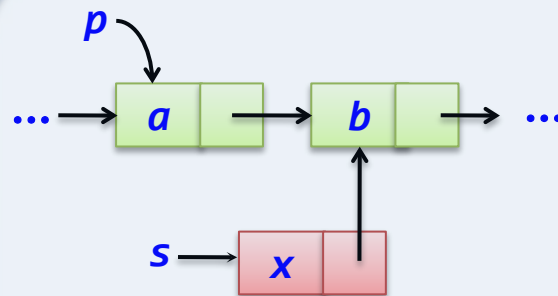


1. 插入和删除结点操作

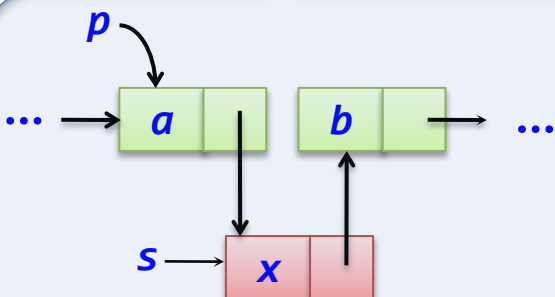
插入结点操作：将结点 s 插入到单链表中 p 结点的后面。



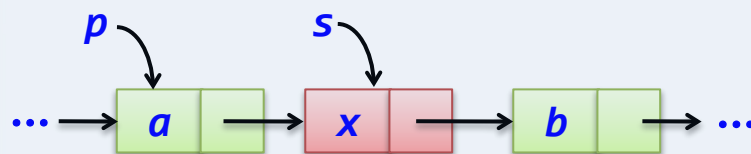
(a) 插入前



(b) $s \rightarrow \text{next} = p \rightarrow \text{next}$



(c) $p \rightarrow \text{next} = s$

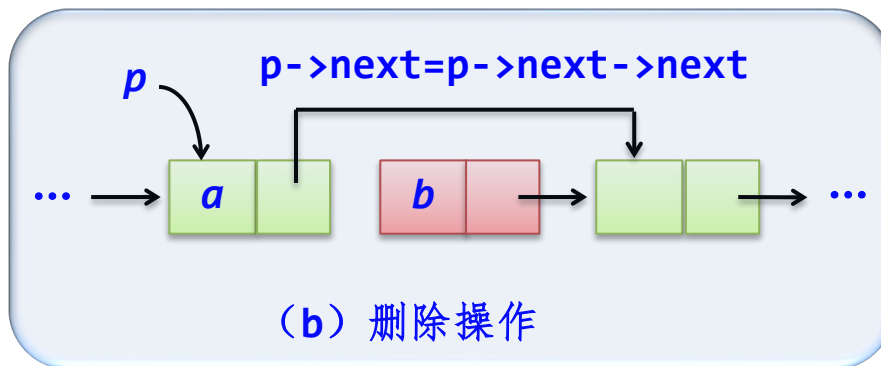
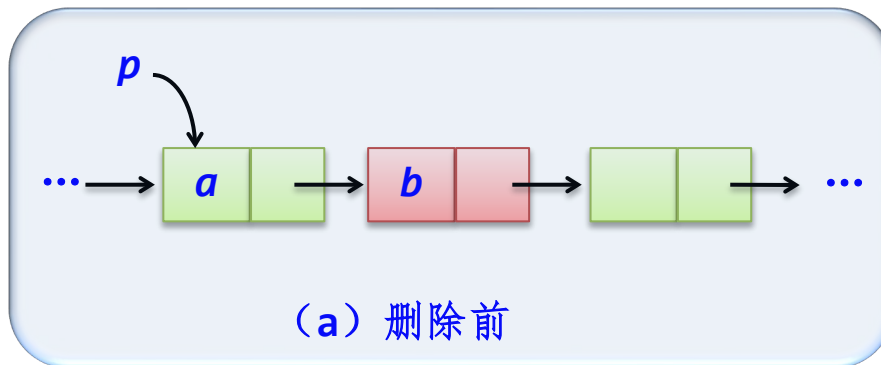


(d) 插入后



```
s->next=p->next;  
p->next=s;
```

删除结点操作：删除单链表中 p 结点的后继结点。



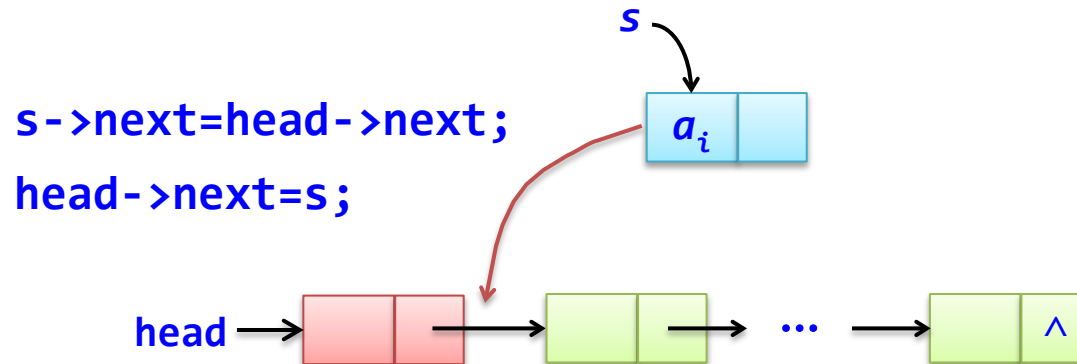
`q=p->next;` //q指向被删结点
`p->next=q->next;` //从单链表中删除结点q
`delete q;` //释放空间

2. 整体建立单链表

- 通过一个含有 n 个元素的 a 数组来建立单链表。
- 建立单链表的常用方法有两种：头插法和尾插法。

头插法建表

- 从一个空表开始，依次读取数组 a 中的元素。
- 生成新结点 s ，将读取的数据存放到新结点的数据成员中。
- 将新结点 s 插入到当前链表的表头上。

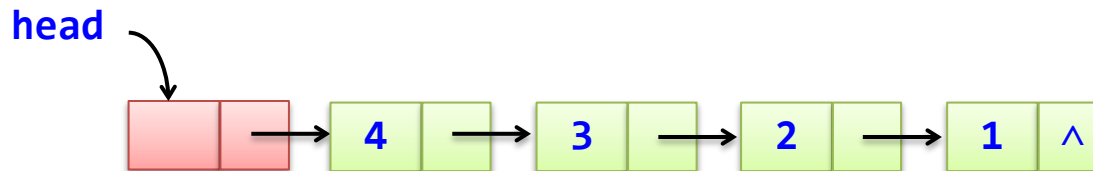


```

void CreateListF(T a[],int n)           //头插法建立单链表
{   for (int i=0;i<n;i++)              //循环建立数据结点
    {   LinkNode<T>* s=new LinkNode<T>(a[i]); //创建数据结点s
        s->next=head->next;              //将结点s插入head结点之后
        head->next=s;
    }
}

```

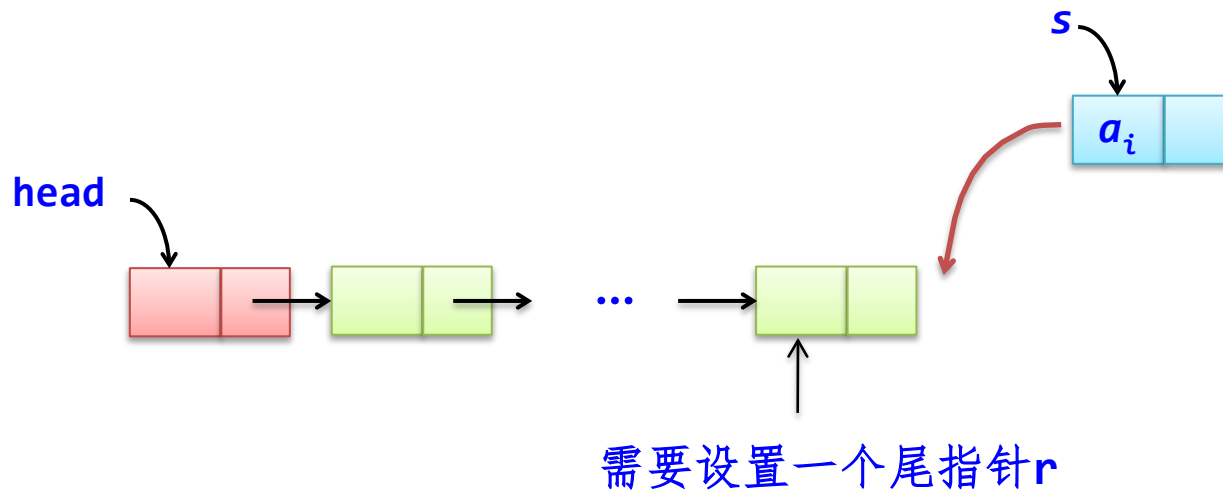
$a=[1,2,3,4]$, 调用CreateListF(a,4)



头插法建立的单链表中数据结点的次序与 a 数组中的次序正好相反

尾插法建表

- 从一个空表开始，依次读取数组 a 中的元素。
- 生成新结点 s ，将读取的数据存放到新结点的数据成员中。
- 将新结点 s 插入到当前链表的表尾上。



```

void CreateListR(T a[],int n)
{  LinkNode<T>* s,*r;
   r=head;
   for (int i=0;i<n;i++)
   {  s=new LinkNode<T>(a[i]);
      r->next=s;
      r=s;
   }
   r->next=NULL;
}

```

//尾插法建立单链表

//r指向尾结点,开始时指向头结点

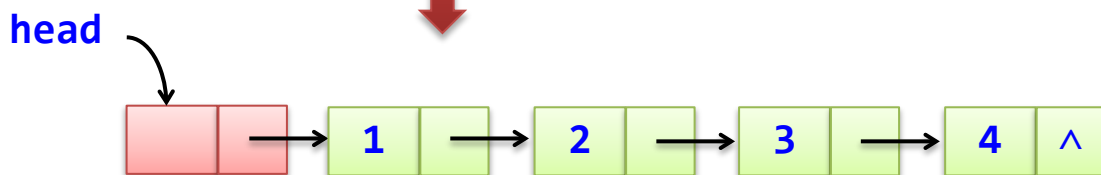
//循环建立数据结点

//创建数据结点s

//将结点s插入结点r之后

//将尾结点的next域置为NULL

$a=[1,2,3,4]$, 调用CreateListR(a,4)

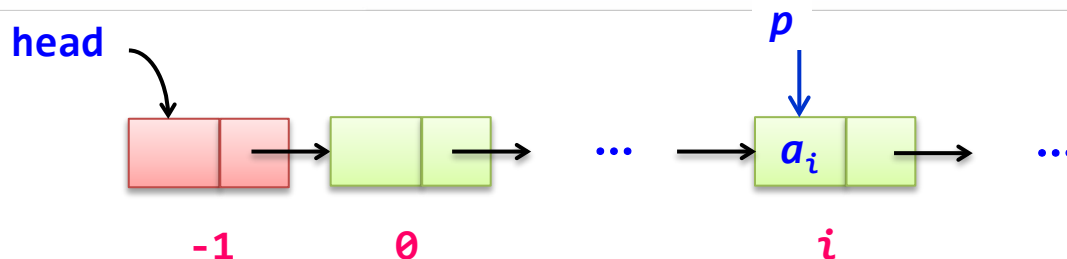


尾插法建立的单链表中数据结点的次序与 a 数组中的次序正好相同

3. 线性表基本运算在单链表中的实现

查找序号为 i ($-1 \leq i \leq n-1$, n 为单链表中数据结点个数) 的结点

```
//*****
//序号i的正确范围:  $-1 \leq i < n$ , 超出范围返回NULL
//i=-1时返回头结点head
//i $\geq 0$ 并且 $i < n$ 时返回序号i的结点
//*****
LinkNode<T>* geti(int i)                //返回序号i的结点
{ if (i<-1) return NULL;                //i<-1返回NULL
  LinkNode<T>* p=head;                  //首先p指向头结点
  int j=-1;                             //j置为-1(可以认为头结点序号为-1)
  while (j<i && p!=NULL)                //指针p移动i+1个结点
  { j++;
    p=p->next;
  }
  return p;                             //返回p
}
```



(1) 单链表的初始化和销毁

构造函数

```
LinkedList()           //构造函数,创建一个空单链表
{
    head=new LinkNode<T>();
}
```

析构函数

~LinkedList()

```
{  ListNode<T>* pre,*p;  
  pre=head;p=pre->next;  
  while (p!=NULL)  
  {  delete pre;  
      pre=p; p=p->next;  
  }  
  delete pre;  
}
```

//析构函数,销毁单链表

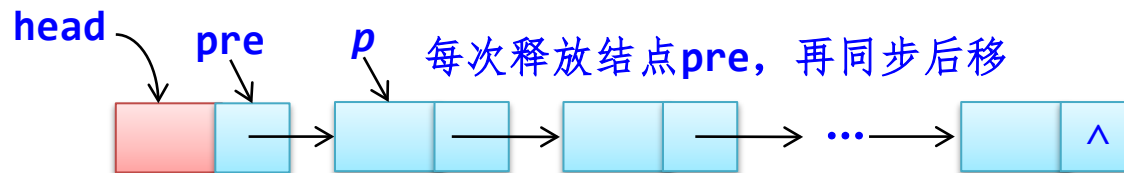
//用p遍历结点并释放其前驱结点

//释放pre结点

//pre,p同步后移一个结点

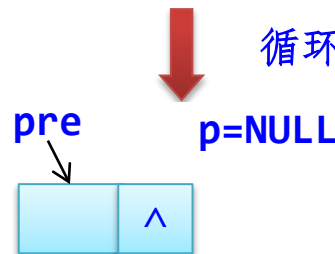
//p为空时pre指向尾结点,此时释放尾结点

while开始前:



循环结束条件是p为空

while结束后:



(2) 将元素 e 添加的线性表末尾Add(e)

```
void Add(T e)                                //在单链表末尾添加一个值为 $e$ 的结点
{
    LinkNode<T>* s=new LinkNode<T>(e);      //新建结点 $s$ 
    LinkNode<T>* p=head;
    while (p->next!=NULL)                    //查找尾结点 $p$ 
        p=p->next;
    p->next=s;                                //在尾结点之后插入结点 $s$ 
}
```



(3) 求线性表的长度Getlength()

```
int Getlength()                                //求单链表中数据结点个数
{
    LinkNode<T>* p=head;
    int cnt=0;
    while (p->next!=NULL)                      //找到尾结点为止
    {
        cnt++;
        p=p->next;
    }
    return cnt;
}
```

思考

若像顺序表中一样，在单链表中设置一个长度length，插入时length++，删除时length--。那么求长度的时间复杂度为 $O(1)$ 了。

(4) 求线性表中序号为*i*的元素GetElem(*i*,&*e*)

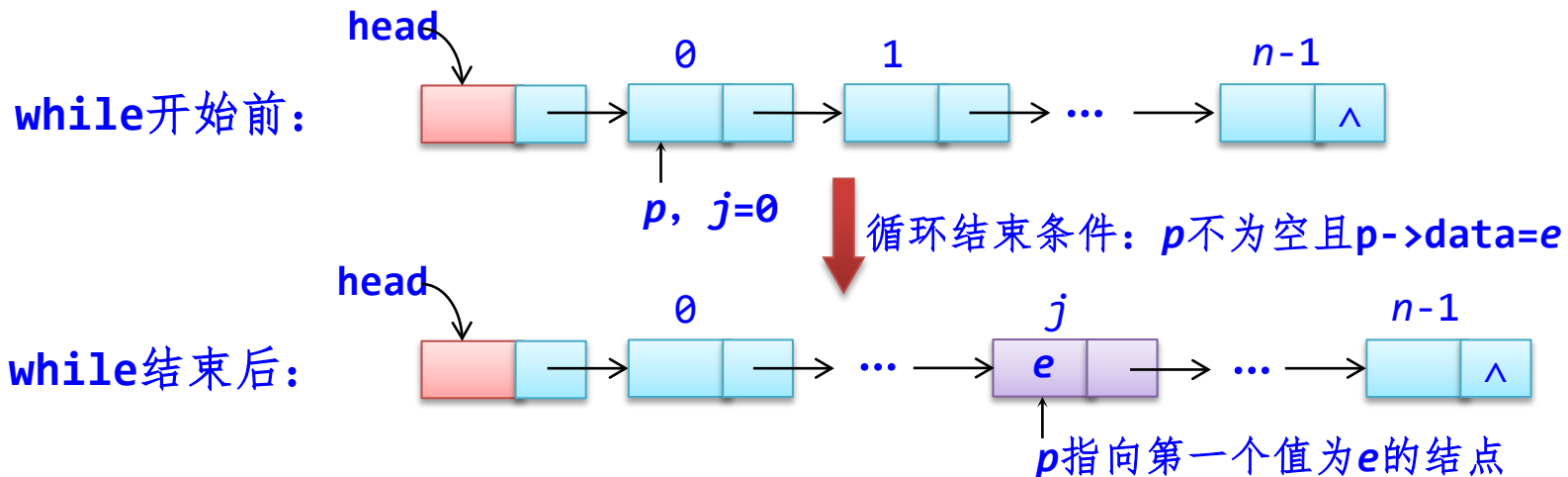
```
bool GetElem(int i,T& e)           //求单链表中序号i的结点值
{  if (i<0) return false;          //参数i错误返回false
    LinkNode<T>* p=geti(i);        //查找序号i的结点
    if (p!=NULL)                   //找到了序号i的结点p
    {  e=p->data;                   //成功找到返回true
        return true;
    }
    else                             //不存在序号i的结点
        return false;              //参数i错误返回false
}
```

(5) 设置线性表中序号为*i*的元素SetElem(*i*, *e*)

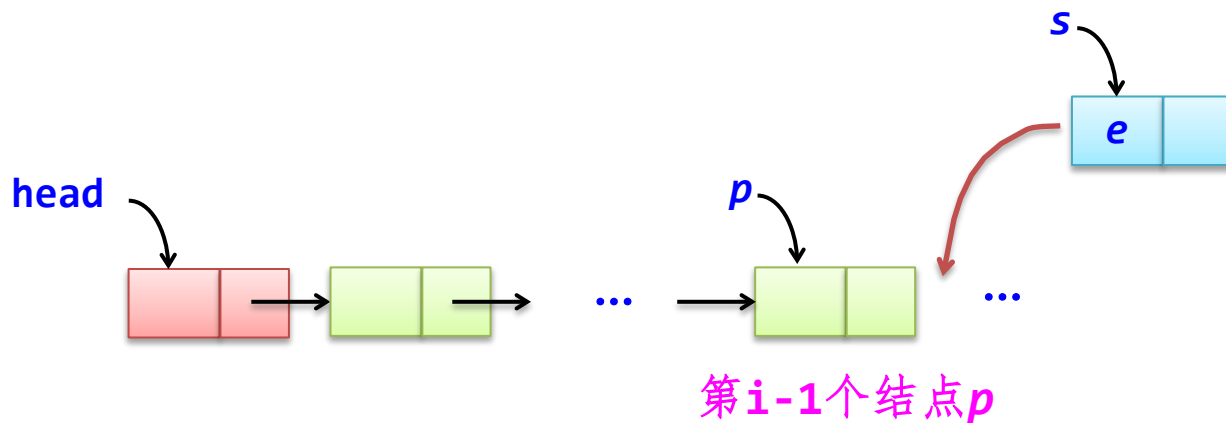
```
bool SetElem(int i, T e)           //设置序号i的结点值
{   if (i<0) return false;         //参数i错误返回false
    LinkNode<T>* p=geti(i);        //查找序号i的结点
    if (p!=NULL)                   //找到了序号i的结点p
    {   p->data=e;
        return true;
    }
    else                             //不存在序号i的结点
        return false;              //参数i错误返回false
}
```

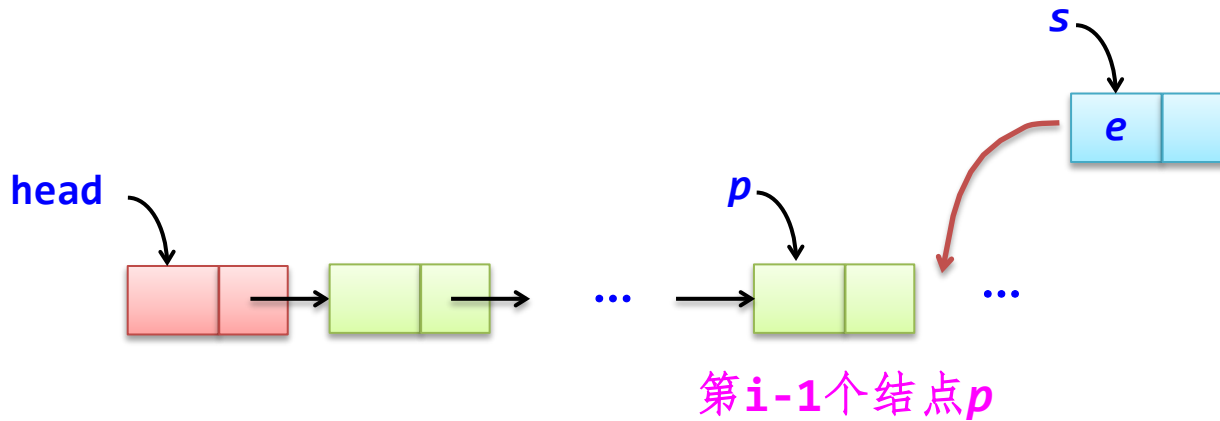
(6) 求线性表中第一个值为 e 的元素的逻辑序号GetNo(e)

```
int GetNo(T e)                                //查找第一个为 $e$ 的元素的序号
{  int j=0;                                   //j置为0, p指向首结点
  LinkNode<T>* p=head->next;
  while (p!=NULL && p->data!=e)              //查找第一个值为 $e$ 的结点p
  {  j++;
    p=p->next;
  }
  if (p==NULL) return -1;                     //未找到时返回-1
  else return j;                             //找到后返回其序号
}
```



(7) 在线性表中插入 e 作为第 i 个元素 $\text{Insert}(i, e)$



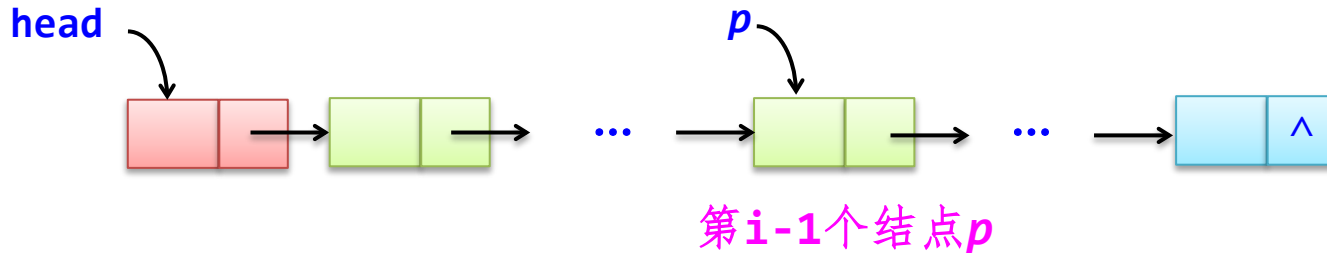


```

bool Insert(int i, T e)
{
    if (i < 0) return false;
    LinkNode<T>* s = new LinkNode<T>(e); // 在序号i位置插入值为e的结点
    LinkNode<T>* p = geti(i-1); // 参数i错误返回false
    if (p != NULL) // 建立新结点s
    {
        s->next = p->next; // 查找序号i-1的结点
        p->next = s; // 找到了序号i-1的结点p
        return true; // 在p结点后面插入s结点
    }
    else // 插入成功返回true
    {
        return false; // 没有找到序号i-1的结点
    }
} // 参数i错误返回false

```

(8) 在线性表中删除第 i 个数据元素Delete(i)



先查找第 $i-1$ 个结点 p :

- ① 如果存在结点 p ，若存在后继结点 q ，删除结点 q ，返回**true**；若不存在后继结点 q ，返回**false**
- ② 如果不存在结点 p ，返回**false**

删除第*i*个结点算法



```
bool Delete(int i)
{
    if (i<0) return false;
    LinkNode<T>* p=geti(i-1);
    if (p!=NULL)
    {
        LinkNode<T>* q=p->next;
        if (q!=NULL)
        {
            p->next=q->next;
            delete q;
            return true;
        }
        else
            return false;
    }
    else
        return false;
}
```

//在单链表中删除序号*i*位置的结点
//参数*i*错误返回false
//查找序号*i-1*的结点
//找到了序号*i-1*的结点*p*
//*q*指向序号*i*的结点(被删结点)
//存在序号*i*的结点时删除它
//删除*p*结点的后继结点
//释放空间
//删除成功返回true

//没有找到序号*i*的结点
//参数*i*错误返回false

//没有找到序号*i-1*的结点
//参数*i*错误返回false

(9) 输出线性表所有元素DispList()

```
void DispList()                                //输出单链表所有结点值
{
    LinkNode<T>* p;                            //p指向开始结点
    p=head->next;                              //p不为NULL,输出p结点的data域
    while (p!=NULL)
    {
        cout << p->data << " ";              //p移向下一个结点
        p=p->next;
    }
    cout << endl;
}
```